

WORKSHOP

Node.js

Do Básico ao Avançado

Curso de Extensão em Desenvolvimento de Software

Runtime · Assincronicidade · I/O · Redes · Streams · Estruturas de Dados · Performance

12 Módulos · Documentação Oficial · Livro: Loiane Groner (Estruturas de Dados e Algoritmos com JavaScript)

Sumário

Módulo 1	Runtime Node.js	Event Loop, Call S
Módulo 2	Programação Assíncrona	Callbacks, Promis
Módulo 3	Sistema de Arquivos	fs (callbacks), fs/p
Módulo 4	Terminal e Input	readline, tty, menu
Módulo 5	Redes e HTTP	Conceitos de rede
Módulo 6	Processos e Sistema	child_process, exe
Módulo 7	Streams e Pipeline	Readable, Writabl
Módulo 8	Gerenciamento de Pacotes	npm, package.json
Módulo 9	CLI Profissional	Commander.js, CL
Módulo 10	Sintaxe de Classes e Estruturas Lineares	class, #private, sta
Módulo 11	Estruturas Não-Lineares	BST, Trie, Graph
Módulo 12	Performance e Segurança	worker_threads, p

MÓDULO 1

Runtime Node.js

Event Loop · Call Stack · Microtasks · Macrotasks · process · Módulos

Node.js é um **ambiente de execução JavaScript fora do navegador**, construído sobre o motor V8 do Google Chrome e a biblioteca de I/O assíncrono **libuv**. Diferente de um servidor Apache (que cria uma thread por requisição), o Node usa uma única thread principal para gerenciar milhares de conexões simultâneas, delegando operações de disco e rede para o sistema operacional de forma não-bloqueante. Compreender como isso funciona internamente é a base para escrever código Node eficiente e sem bugs sutis de concorrência.

1.1 — A Arquitetura Interna: V8 + libuv

O Node não é apenas "JavaScript no servidor". Por baixo, dois componentes trabalham juntos de forma complementar:

Componente	Papel	O que faz
V8	Motor JS	Compila e executa JavaScript — gerencia Call Stack, memória (GC) e otimizações JIT
libuv	I/O assíncrono	Gerencia o Event Loop, thread pool para I/O de disco, DNS, e operações de rede
Bindings C++	Ponte	Conectam as APIs do Node (fs, http, crypto) ao sistema operacional
npm	Ecossistema	Gerenciador de pacotes — repositório de mais de 2 milhões de bibliotecas

1.2 — O Event Loop em Detalhes

O **Event Loop** é o mecanismo que faz o Node ser "non-blocking". Enquanto a Call Stack executa código síncrono, o Event Loop monitora filas de callbacks prontos para serem executados. Ele possui **fases bem definidas**, cada uma com sua própria fila:

- **timers** — Executa callbacks de setTimeout() e setInterval() com delay expirado.
- **pending callbacks** — Callbacks de erros de I/O adiados para o ciclo seguinte.
- **idle/prepare** — Uso interno do libuv.
- **poll** — Busca novos eventos de I/O. Aqui o loop aguarda se não há callbacks.
- **check** — Executa callbacks de setImmediate().
- **close callbacks** — Executa callbacks de eventos close (ex: socket.on("close")).

■ Microtasks vs Macrotasks

Entre cada fase do Event Loop, o Node drena completamente a fila de **microtasks** (Promises resolvidas e `queueMicrotask`). Isso significa que microtasks têm prioridade sobre todas as fases do Event Loop. Macrotasks (`setTimeout`, `setInterval`, `setImmediate`, I/O) são processadas uma por fase. Essa distinção é fundamental para entender a ordem de execução do código assíncrono.

```
// Demonstração da ordem de execução do Event Loop
console.log("1 - síncrono: Call Stack");

setTimeout(() => console.log("5 - macrotask: setTimeout 0ms"), 0);

setImmediate(() => console.log("6 - check phase: setImmediate"));

Promise.resolve()
  .then(() => console.log("3 - microtask: Promise.then"));

queueMicrotask(() => console.log("4 - microtask: queueMicrotask"));

console.log("2 - síncrono: ainda na Call Stack");
```

■ Output

```
1 - síncrono: Call Stack
2 - síncrono: ainda na Call Stack
3 - microtask: Promise.then
4 - microtask: queueMicrotask
5 - macrotask: setTimeout 0ms
6 - check phase: setImmediate
```

O código síncrono sempre executa primeiro. Depois, microtasks esgotam a fila. Por fim, macrotasks são processadas uma por ciclo.

1.3 — Call Stack e Non-Blocking I/O

A **Call Stack** é uma estrutura LIFO (Last In, First Out) que rastreia onde o programa está na execução. Quando você chama uma função, ela entra na stack. Quando retorna, sai. Se a stack não esvaziar (ex: loop infinito ou recursão sem base), o processo trava — isso se chama **stack overflow**.

Operações de I/O (ler arquivo, fazer request HTTP) seriam desastrosas na Call Stack pois podem levar segundos. O Node delega essas operações ao libuv/SO, que as executa em background e coloca o callback na fila quando prontas — nunca bloqueando a thread principal.

■ Output

1.4 — Módulos: CommonJS vs ES Modules

```
// ■■■ CommonJS (CJS) – padrão histórico ██████████  
// math.js  
  
function soma(a, b) { return a + b; }  
  
const PI = 3.14159;  
  
module.exports = { soma, PI };      // Exportar  
  
// app.js  
  
const { soma, PI } = require("./math"); // Importar  
  
const fs           = require("fs");     // Módulo nativo  
  
console.log(soma(2, 3)); // 5  
  
// ■■■ ES Modules (ESM) – padrão moderno ██████████  
// math.mjs OU package.json com "type": "module"  
  
export function soma(a, b) { return a + b; } // Named export  
  
export const PI = 3.14159;  
  
export default class Calculadora { /* ... */ } // Default export  
  
// app.mjs  
  
import Calculadora, { soma, PI } from "./math.mjs";  
  
import { readFile }          from "fs/promises"; // Nativo ESM  
  
import path, { join }       from "path";  
  
// Top-level await – só funciona em ESM  
  
const dados = await readFile("config.json", "utf-8");
```

■ `__dirname` e `__filename` em ESM

Em CommonJS, `__dirname` e `__filename` são variáveis globais disponíveis automaticamente. Em ES Modules, elas não existem. Use o padrão abaixo para obtê-las: `import { fileURLToPath } from "url"; import path from "path"; const __filename = fileURLToPath(import.meta.url); const __dirname = path.dirname(__filename);`

1.5 — O Objeto `process`

O objeto global `process` é a interface entre o script Node e o processo do sistema operacional. Ele é global (não precisa de `import`) e é fundamental para ferramentas de linha de comando.

- Documentação oficial — process
- Documentação oficial — ES Modules
- Documentação oficial — Módulos CJS
- Visualizador do Event Loop (Philip Roberts)
 - *The Node.js Event Loop, Timers, and process.nextTick()* — Guia oficial aprofundado

MÓDULO 2

Programação Assíncrona

Callbacks → Promises → async/await · Error Handling · Promise combinators

A programação assíncrona é o coração do Node.js. Ela permite que seu programa **inicie uma operação demorada e continue executando** enquanto aguarda o resultado, em vez de travar e esperar. Neste módulo, você aprenderá a evolução histórica dos padrões assíncronos em JavaScript — cada um resolvendo as limitações do anterior.

2.1 — Callbacks: O Ponto de Partida

O padrão mais antigo de assincronicidade em JavaScript é o **callback**: uma função passada como argumento que será chamada quando a operação terminar. Node.js popularizou o padrão **error-first callback**, onde o primeiro argumento do callback é sempre um erro (ou null se tudo correu bem).

```
// Padrão error-first callback (Node.js convention)
// Assinatura: callback(err, resultado)

const fs = require("fs");

fs.readFile("dados.json", "utf-8", function(err, conteudo) {
  if (err) {
    console.error("Erro ao ler:", err.message);
    return; // SEMPRE retornar após tratar o erro
  }
  const dados = JSON.parse(conteudo);
  console.log(dados);
});

console.log("Isso executa antes do callback!");
```

■ Output

```
Isso executa antes do callback!
{ nome: "Node.js", versao: "20" }
```

■ Callback Hell — O Problema dos Callbacks Aninhados

Quando você precisa executar operações assíncronas em sequência, os callbacks se aninham, criando o temido "Pyramid of Doom":
`lerArquivo(path, (err, a) => { processarDados(a, (err, b) => {
 salvarResultado(b, (err, c) => { enviarNotificacao(c, (err) => { // Quatro níveis de indentação e 4 handlers de erro separados! }); }); }); });`
Promises foram criadas exatamente para resolver este problema.

2.2 — Promises: Controle do Fluxo Assíncrono

Uma **Promise** é um objeto que representa o resultado futuro de uma operação assíncrona. Ela pode estar em um de três estados: **pending** (aguardando), **fulfilled** (concluída com sucesso) ou **rejected** (concluída

com erro). Uma vez que muda de estado, nunca volta — é imutável.

```
// ■■■ Criando uma Promise ■■■■
```

```
function lerArquivoPromise(caminho) {  
    return new Promise((resolve, reject) => {  
        // resolve(valor) → fulfills a Promise  
        // reject(erro) → rejects a Promise  
        fs.readFile(caminho, "utf-8", (err, dados) => {  
            if (err) reject(err);      // Propaga o erro  
            else     resolve(dados);   // Entrega o valor  
        });  
    });  
  
}  

```

```
// ■■■ Consumindo com .then / .catch / .finally ■■■■
```

```
lerArquivoPromise("dados.json")  
    .then(conteudo => JSON.parse(conteudo)) // Transforma o valor  
    .then(dados    => console.log(dados))   // Usa o valor transformado  
    .catch(err     => console.error(err))   // Captura qualquer erro acima  
    .finally(()    => console.log("Fim!")); // Sempre executa
```

Note que `.then()` pode receber até dois callbacks: o primeiro para sucesso e o segundo para erro. Mas o padrão recomendado é usar `.catch()` separado, pois ele captura erros de qualquer `.then()` anterior na cadeia.

```
// ■■■ Encadeamento (chaining) ■■■■  
// Sem callback hell! Cada .then retorna uma nova Promise  
  
lerArquivoPromise("config.json")  
    .then(texto => JSON.parse(texto))  
    .then(config => buscarDadosDaAPI(config.url))  
    .then(dados => processarDados(dados))  
    .then(result => salvar("output.json", result))  
    .then(() => console.log("Pipeline concluída!"))  
    .catch(err => {  
        // Captura qualquer erro em qualquer .then() acima  
        console.error("Falha na pipeline:", err.message);  
    })  
    .finally(() => {  
        // Executa independente de sucesso ou falha  
        fecharConexoes();  
    });
```

2.3 — async/await: Código Assíncrono com Aparência Síncrona

async/await é **açúcar sintático** sobre Promises — não é um novo mecanismo, mas uma forma mais legível de escrever o mesmo código. Uma função async sempre retorna uma Promise. await pausa a execução da função async até a Promise resolver, sem bloquear a thread principal.

```
// A mesma pipeline, com async/await
async function executarPipeline() {
  const texto = await lerArquivoPromise("config.json");
  const config = JSON.parse(texto);
  const dados = await buscarDadosDaAPI(config.url);
  const result = await processarDados(dados);
  await salvar("output.json", result);
  console.log("Pipeline concluída!");
}

// Chamar uma função async: ela retorna uma Promise
executarPipeline(); // sem await = "fire and forget"

// Na prática, você também pode aguardar a chamada:
await executarPipeline(); // Só funciona em contexto async ou top-level ESM
```

2.4 — Error Handling com try/catch

Com async/await, o tratamento de erros volta ao padrão síncrono do JavaScript: try/catch. Isso é uma das grandes vantagens — o fluxo de erro fica visualmente próximo do código que pode falhar.

```
async function carregarConfiguracoes() {
  try {
    const texto = await lerArquivoPromise("config.json");
    const config = JSON.parse(texto); // Pode lançar SyntaxError
    return config;
  } catch (err) {
    // Captura tanto erros de I/O quanto de JSON.parse
    if (err.code === "ENOENT") {
      console.warn("config.json não encontrado, usando padrões.");
      return { porta: 3000, debug: false }; // Valor padrão
    }
    throw err; // Re-lança erros que não sabemos tratar
  } finally {
    // Sempre executa — ótimo para limpeza de recursos
    console.log("Tentativa de carregamento concluída.");
  }
}
```

■ Erro Comum: Esquecer de await

Um dos erros mais frequentes com async/await é esquecer o await. Sem ele, você recebe a Promise em si, não o valor resolvido: `const dados = lerArquivoPromise("x.json");` // ■ Promise, não string! `const dados = await lerArquivoPromise("x.json");` // ■ O TypeScript detecta esse erro em tempo de compilação — mais um motivo para adotá-lo em projetos maiores.

2.5 — Promise Combinators: Paralelismo Controlado

Às vezes você precisa executar múltiplas operações assíncronas **ao mesmo tempo**. Os combinadores de Promise permitem isso de formas diferentes:

```
// Promise.all – aguarda TODAS. Falha se qualquer uma falhar
const [usuarios, posts, config] = await Promise.all([
  buscarUsuarios(),
  buscarPosts(),
  lerConfig(),
]);

// Promise.allSettled – aguarda TODAS, independente de falhas
// Retorna: [{ status: "fulfilled", value }, { status: "rejected", reason }]
const resultados = await Promise.allSettled([
  sincronizarAPI(),
  fazerBackup(),
  enviarRelatorio(),
]);
const falhas = resultados.filter(r => r.status === "rejected");

// Promise.race – retorna a PRIMEIRA a resolver (ou rejeitar)
const resposta = await Promise.race([
  fetchDados("https://api1.com"),
  fetchDados("https://api2.com"), // API espelho
]);

// Promise.any – retorna a PRIMEIRA a resolver com sucesso
const resultado = await Promise.any([
  tentarServidor("primario"),
  tentarServidor("secundario"),
  tentarServidor("terciario"),
]); // Só rejeita se TODAS falharem (AggregateError)
```

Promise.all é ideal para operações independentes que podem ser paralelizadas. Promise.allSettled é preferível quando você quer tratar falhas individualmente.

2.6 — Convertendo Callbacks em Promises: util.promisify

Boa parte das APIs do Node usa error-first callbacks. O utilitário `util.promisify` converte qualquer função que segue essa convenção em uma função que retorna Promise — sem reescrever o código.

```
import { promisify } from "util";
import { exec } from "child_process";
import fs from "fs";

// Converter exec para Promise
const execAsync = promisify(exec);

// Agora pode usar com await
const { stdout } = await execAsync("git log --oneline -5");
console.log(stdout);

// fs.readFile também pode ser promisifyado manualmente
const readFileAsync = promisify(fs.readFile);
const conteudo = await readFileAsync("dados.txt", "utf-8");

// Mas prefira fs/promises para funções do fs
import { readFile } from "fs/promises"; // Mais idiomático em ESM
```

■ [MDN — Usando Promises](#)

■ [MDN — async/await](#)

■ [Documentação oficial — util.promisify](#)

→ [JavaScript.info — Promises, async/await](#) — Guia progressivo de alta qualidade

→ [Node.js Best Practices — Error Handling](#) — Repositório de referência

MÓDULO 3

Sistema de Arquivos

fs (callbacks) · fs/promises (async/await) · path · watchers

O módulo fs (File System) é a interface do Node com o sistema de arquivos. Ele oferece **duas APIs paralelas** para as mesmas operações: uma baseada em callbacks (API clássica) e outra baseada em Promises (fs/promises). Ambas são importantes: você encontrará callbacks em código legado e em situações onde o estilo é mais natural; Promises/async-await são o padrão para código novo.

3.1 — API de Callbacks (fs) — O Estilo Clássico

A API de callbacks do fs segue sempre o padrão error-first. É importante conhecê-la porque é a que aparece na maioria dos exemplos históricos, tutoriais antigos e código legado. Também é útil quando você está dentro de um contexto que não suporta async/await.

```

const fs = require("fs"); // CommonJS
// import fs from "fs"; // ESM

// ■■■■ Leitura ■■■■
fs.readFile("package.json", "utf-8", (err, conteudo) => {
  if (err) {
    if (err.code === "ENOENT") console.error("Arquivo não encontrado");
    else console.error("Erro desconhecido:", err);
    return;
  }
  const pkg = JSON.parse(conteudo);
  console.log("Nome:", pkg.name);
});

// ■■■■ Escrita ■■■■
const novoConteudo = JSON.stringify({ chave: "valor" }, null, 2);
fs.writeFile("saida.json", novoConteudo, "utf-8", (err) => {
  if (err) { console.error("Erro ao salvar:", err.message); return; }
  console.log("Arquivo salvo com sucesso!");
});

// ■■■■ Verificar existência ■■■■
fs.access("config.json", fs.constants.F_OK, (err) => {
  if (err) console.log("Arquivo não existe");
  else console.log("Arquivo existe");
});

// ■■■■ Leitura SÍNCRONA – bloqueante, evitar em servidores ■■■■
try {
  const dados = fs.readFileSync("config.json", "utf-8");
  const config = JSON.parse(dados);
} catch (err) {
  if (err.code === "ENOENT") { /* arquivo não existe */ }
}

```

◆ Quando usar a versão síncrona (readFileSync, writeFileSync)?

A versão síncrona bloqueante é aceitável apenas em dois cenários: (1) Scripts de linha de comando simples que não precisam ser concorrentes; (2) Inicialização de configurações no startup do processo, antes de qualquer requisição ser aceita. Em servidores e sistemas de alta carga, nunca use a versão síncrona — ela trava toda a thread e prejudica todos os usuários.

3.2 — API de Promises (fs/promises) — O Estilo Moderno

O submódulo fs/promises (disponível desde Node 10, estável a partir do Node 14) oferece as mesmas operações do fs, mas todas retornam Promises, permitindo uso natural com async/await.

[illegible]

```
// ■■■ Escrita segura (write → rename atômico) ■■■
```

[illegible]

```
// ■■■ Metadatos de archivo ■■■■
```

[illegible]

3.4 — Watchers — Monitorar Mudanças em Tempo Real

O `fs.watch` monitora arquivos e diretórios por mudanças, chamando um callback quando detecta alteração. Isso é a base de ferramentas como o `nodemon` e sistemas de `hot-reload`.


```
import { watch } from "fs";

// Monitorar um arquivo específico
const watcher = watch("./data/db.json", { encoding: "utf-8" }, (evento, nome) => {
  console.log(`[${evento}] ${nome}`);
  // eventos: "rename" (criado/deletado) ou "change" (modificado)
});

// Monitorar diretório inteiro (recursivo)
watch("./src", { recursive: true }, (evento, caminho) => {
  if (caminho?.endsWith(".js")) {
    console.log(`Arquivo JS modificado: ${caminho}`);
    recarregarModulo(caminho);
  }
});

// Encerrar o watcher quando não precisar mais
process.on("SIGINT", () => {
  watcher.close();
  process.exit(0);
});
```

- [Documentação oficial — fs](#)
- [Documentação oficial — fs/promises](#)
- [Documentação oficial — path](#)

MÓDULO 4

Terminal e Input

readline · tty · menus interativos · SIGINT

O módulo `readline` permite ler entrada do usuário linha por linha a partir de qualquer stream — normalmente `process.stdin`. É a base de qualquer interface interativa nativa no terminal.

```
import readline from "readline";

// Criar interface de leitura
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

// Wrapper Promise para rl.question()
const perguntar = (texto) => new Promise(resolve => rl.question(texto, resolve));

// Menu interativo com loop recursivo
async function menu() {
  console.log("\n■■■■ DevTrack ■■■■");
  console.log("1 — Adicionar tarefa");
  console.log("2 — Listar tarefas");
  console.log("0 — Sair");

  const opcao = await perguntar("\nEscolha: ");

  switch (opcao.trim()) {
    case "1": await adicionarTarefa(); return menu();
    case "2": await listarTarefas(); return menu();
    case "0": rl.close(); process.exit(0);
    default: console.log("Opção inválida."); return menu();
  }
}

// Capturar Ctrl+C antes do menu abrir
process.on("SIGINT", () => { rl.close(); process.exit(0); });

await menu();
```

Note que wrappamos `rl.question` em uma `Promise` para usar com `await` — padrão muito comum ao trabalhar com `readline`.

4.1 — tty — Detectar o Ambiente do Terminal

```

// process.stdout.isTTY é true somente em terminal interativo real
// false quando a saída é redirecionada para arquivo ou pipe

if (process.stdout.isTTY) {
  // Modo interativo – pode usar cores e formatação
  const { columns, rows } = process.stdout;
  console.log(`Terminal: ${columns} colunas x ${rows} linhas`);
} else {
  // Modo não-interativo – pipe ou redirect
  // Saída deve ser limpa (sem cores, sem caracteres especiais)
  console.log(JSON.stringify(dados)); // Mais seguro para scripts
}

// Uso combinado com o módulo anterior (Módulo 3)
// Detecta o ambiente e decide o formato de saída
async function listarTarefas() {
  const db = await lerDB(); // Módulo 3: fs/promises
  if (!process.stdout.isTTY) {
    process.stdout.write(JSON.stringify(db.tasks) + "\n");
    return;
  }
  // Terminal: exibe tabela formatada com alinhamento
  console.log("ID      TÍTULO      STATUS");
  for (const t of db.tasks) {
    console.log(`${t.id.slice(0,8)}  ${t.titulo.padEnd(20)}  ${t.status}`);
  }
}

```

- [Documentação oficial — readline](#)
- [Documentação oficial — tty](#)

MÓDULO 5

Redes e HTTP

Conceitos de rede · http/https · fetch · url · dns

Antes de escrever qualquer código de rede, é fundamental compreender os conceitos que estão por baixo. A tabela abaixo apresenta os termos essenciais. Para cada um, indicamos fontes de alta qualidade para você aprofundar — pesquisar e ler sobre esses conceitos é parte do exercício.

5.1 — Conceitos Fundamentais de Rede

Conceito	O que é	Para aprofundar
HTTP/HTTPS	Protocolo de transferência de hipertexto. Defines como cliente e servidor trocam mensagens. HTTPS adiciona criptografia TLS sobre o HTTP.	MDN — Visão geral do HTTP
URL	Uniform Resource Locator. Endereço completo de um recurso: scheme://user:pass@host:port/path?query#hash	MDN — O que é uma URL?
DNS	Domain Name System. Traduz nomes legíveis (google.com) em endereços IP. É a "agenda telefônica" da internet.	Cloudflare — O que é DNS?
Domínio e Subdomínio	Domínio: github.com. Subdomínio: api.github.com. O subdomínio aponta para um servidor ou serviço diferente.	MDN — Nomes de domínio
TCP/IP	Protocolos de transporte da internet. TCP garante entrega ordenada e confiável de pacotes; IP cuida do roteamento.	Cloudflare — O que é TCP/IP?
Porta	Número que identifica um serviço específico em um host. HTTP=80, HTTPS=443, Node local=3000, PostgreSQL=5432.	MDN — Portas TCP e UDP
Status HTTP	1xx=informativo, 2xx=sucesso, 3xx=redirecionamento, 4xx=erro do cliente, 5xx=erro do servidor.	MDN — Códigos de status HTTP
Headers	Metadados enviados junto com requisições e respostas: Content-Type, Authorization, Cache-Control, etc.	MDN — HTTP Headers

5.2 — Servidor HTTP com o Módulo http

```

import http from "http";
import { readFile } from "fs/promises"; // Módulo 3

const server = http.createServer(async (req, res) => {
  // Parsear URL para extrair rota e query params
  const url = new URL(req.url, `http://${req.headers.host}`);
  const rota = url.pathname;
  const metodo = req.method;

  res.setHeader("Content-Type", "application/json; charset=utf-8");

  // Roteamento manual
  if (metodo === "GET" && rota === "/health") {
    res.writeHead(200);
    return res.end(JSON.stringify({ status: "ok", uptime: process.uptime() }));
  }

  if (metodo === "GET" && rota === "/tasks") {
    try {
      const db = JSON.parse(await readFile("./data/db.json", "utf-8"));
      res.writeHead(200);
      return res.end(JSON.stringify(db.tasks));
    } catch (err) {
      res.writeHead(500);
      return res.end(JSON.stringify({ error: err.message }));
    }
  }

  // Rota não encontrada
  res.writeHead(404);
  res.end(JSON.stringify({ error: "Not Found" }));
});

server.listen(3000, "127.0.0.1", () => {
  console.log("Servidor em http://127.0.0.1:3000");
});

```

5.3 — Cliente HTTP: fetch Nativo (Node 18+)

```
// fetch é global a partir do Node 18 — sem necessidade de import

async function buscarIssuesGitHub(repo, token) {
  const url = `https://api.github.com/repos/${repo}/issues?state=open`;

  const resposta = await fetch(url, {
    headers: {
      "Authorization": `Bearer ${token}`,
      "Accept": "application/vnd.github.v3+json",
      "User-Agent": "DevTrack/1.0",
    },
  });

  // Verificar status antes de parsear
  if (resposta.status === 401) throw new Error("Token inválido ou expirado");
  if (resposta.status === 404) throw new Error(`Repositório "${repo}" não encontrado`);
  if (!resposta.ok) throw new Error(`HTTP ${resposta.status}: ${resposta.statusText}`);

  return resposta.json(); // Retorna Promise<objeto>
}

// Uso com Módulos 2 + 3
try {
  const issues = await buscarIssuesGitHub("microsoft/vscode", process.env.GITHUB_TOKEN);
  console.log(`${issues.length} issues abertas`);
} catch (err) {
  console.error("Falha na busca:", err.message);
}
```

Combinando com Módulos 2 (async/await, try/catch) e 3 (process.env para o token).

■ [Documentação oficial — http](#)

■ [Documentação oficial — url \(URL class\)](#)

■ [Documentação oficial — dns](#)

■ [MDN — Fetch API](#)

→ [Curso: How the Internet Works \(Khan Academy\)](#) — Gratuito e visual — conceitos de rede do zero

→ [HTTP Cats — Status codes de forma visual](#) — Referência rápida e memorável para status HTTP

MÓDULO 6

Processos e Sistema

child_process · exec · spawn · os · informações do SO

O módulo `child_process` permite que seu script Node execute outros programas — comandos shell, outros scripts, executáveis do sistema. É a espinha dorsal de automações, scripts de build e integrações com ferramentas de terceiros.

6.1 — exec vs spawn: Quando Usar Cada Um

Método	Comportamento	Usar quando
<code>exec()</code>	Buffer toda a saída na memória. Retorna stdout/stderr ao final.	Comandos curtos com saída pequena (< 200MB).
<code>execSync()</code>	Igual ao <code>exec</code> , mas síncrono. Bloqueia a thread.	Scripts simples, inicialização, configuração pontual.
<code>spawn()</code>	Streams de I/O em tempo real. Não bufferiza.	Processos longos com muita saída (build, ffmpeg, git clone).
<code>execFile()</code>	Como <code>exec</code> , mas sem shell — mais seguro.	Quando precisar de performance e segurança (sem injeção de shell).
<code>fork()</code>	Cria processo Node filho com canal IPC embutido.	Comunicação bidirecional com processos Node filhos.

[illegible]

Usando `util.promisify` (Módulo 2) para converter `exec` em função `async`. Usando `process.cwd()` (Módulo 1) para garantir o diretório correto.

- Documentação oficial — `child_process`
- Documentação oficial — `os`

Streams e Pipeline

Readable · Writable · Transform · pipeline · casos reais · benchmark

Streams são um dos conceitos mais poderosos e subutilizados do Node.js. Elas permitem processar dados **em pedaços (chunks)**, sem precisar carregar o arquivo inteiro na memória. Isso é fundamental para sistemas que lidam com grandes volumes de dados.

7.1 — O Princípio das Streams no Mundo Real

O conceito de stream não é exclusivo do Node — ele está em toda parte:

- **Netflix/YouTube:** Você começa a assistir o vídeo antes de ele ser completamente baixado. O servidor envia chunks enquanto você assiste.
- **Google Drive:** Upload de um arquivo de 10GB não precisa de 10GB de RAM no servidor. Os dados chegam em chunks e são gravados em disco progressivamente.
- **Compiladores:** Ler e parsear um arquivo de código-fonte linha por linha, sem carregar todo o arquivo na memória antes de processar.
- **Logs de servidor:** Analisar um arquivo de log de 50GB processando linha por linha com uso mínimo de memória.
- **Criptografia em tempo real:** Cifrar dados à medida que chegam, sem esperar o arquivo completo.

7.2 — Benchmark: Streams vs Carregamento em Memória

Para um arquivo de **500MB** de logs, veja a diferença prática entre os dois abordagens:

```
// ■ SEM STREAM — carrega TUDO na memória
async function contarLinhasSemStream(arquivo) {
  const inicio = performance.now();
  const texto = await readFile(arquivo, "utf-8"); // RAM: ~500MB
  const linhas = texto.split("\n").length;
  const tempo = performance.now() - inicio;
  console.log(`${linhas} linhas em ${tempo.toFixed(0)}ms | RAM: ~500MB`);
}

// ■ COM STREAM — processa chunk por chunk
async function contarLinhasComStream(arquivo) {
  const inicio = performance.now();
  let linhas = 0;
  let restante = "";

  const stream = createReadStream(arquivo, { encoding: "utf-8" });

  for await (const chunk of stream) { // Async iteration sobre Readable
    const partes = (restante + chunk).split("\n");
    restante = partes.pop(); // Última parte pode estar incompleta
    linhas += partes.length;
  }
  if (restante.length) linhas++; // Última linha sem \n

  const tempo = performance.now() - inicio;
  console.log(`${linhas} linhas em ${tempo.toFixed(0)}ms | RAM: ~64KB`);
}
```

■ Output

```
// Arquivo: logs-servidor.txt (500MB)
Sem stream: 8.240.000 linhas em 4800ms | RAM: ~500MB
Com stream: 8.240.000 linhas em 1200ms | RAM: ~64KB

// Stream é 4x mais rápida E usa 7.800x menos memória
```

O tamanho do chunk padrão é 64KB. Isso significa que o processo nunca usa mais do que ~128KB de RAM, independente do tamanho do arquivo.

7.3 — Os 4 Tipos de Stream

```
import { Readable, Writable, Transform, Duplex } from "stream";
import { createReadStream, createWriteStream } from "fs";
import { pipeline } from "stream/promises";
import zlib from "zlib";

// ■■■ pipeline: encadeia streams de forma segura ■■■■■■■■■■■■■■■■■■■■
// Trata erros automaticamente. Se um stream falhar, os outros são fechados.
await pipeline(
    createReadStream("logs/app.log"), // Readable: lê o arquivo
    zlib.createGzip(),                // Transform: comprime
    createWriteStream("logs/app.log.gz"), // Writable: escreve
);
console.log("Arquivo comprimido com sucesso!");
```

7.4 — Transform Stream: Transformar Dados em Fluxo

```
// Transform personalizado: converte CSV em objetos JSON
import { Transform } from "stream";

class CSVParaJSON extends Transform {
  constructor() {
    super({ objectMode: true }); // Produz objetos, não buffers
    this._headers = null;
    this._buffer = "";
  }

  _transform(chunk, _encoding, callback) {
    this._buffer += chunk.toString();
    const linhas = this._buffer.split("\n");
    this._buffer = linhas.pop(); // Guarda linha incompleta para próximo chunk

    for (const linha of linhas) {
      if (!linha.trim()) continue; // Pular linhas vazias

      if (!this._headers) {
        this._headers = linha.split(",").map(h => h.trim());
      } else {
        const valores = linha.split(",");
        const obj = Object.fromEntries(
          this._headers.map((h, i) => [h, valores[i]??.trim()])
        );
        this.push(obj); // Emite o objeto para o próximo stream
      }
    }
    callback(); // Sinaliza que o chunk foi processado
  }

  _flush(callback) {
    // Processa o que restou no buffer ao final do stream
    if (this._buffer.trim() && this._headers) {
      const valores = this._buffer.split(",");
      this.push(Object.fromEntries(
        this._headers.map((h, i) => [h, valores[i]??.trim()])
      ));
    }
    callback();
  }
}

// Uso: processar CSV de 2GB com uso mínimo de memória
const parser = new CSVParaJSON();

await pipeline(
  createReadStream("tarefas-exportadas.csv"),
  parser,

```

```
async function* (source) { // Async generator como Writable
  for await (const tarefa of source) {
    await salvarNoDB(tarefa); // Módulo 3: fs/promises
  }
}
);
```

■ [Documentação oficial — stream](#)

■ [Documentação oficial — zlib](#)

■ [Documentação oficial — stream/promises](#)

→ [Node.js Streams — Guia Definitivo](#) — *Guia oficial sobre backpressure em streams*

MÓDULO 8

Gerenciamento de Pacotes

npm · package.json completo · semver · scripts · publicação

O npm (Node Package Manager) é o maior repositório de software do mundo. Dominar o package.json vai além de listar dependências: ele é o manifesto completo do seu projeto. Vamos explorar cada campo relevante.

```
{
  "name": "devtrack",           // Nome único no npm (minúsculas, hífens)
  "version": "1.0.0",          // SemVer: MAJOR.MINOR.PATCH
  "description": "CLI para gerenciamento de projetos",
  "type": "module",            // "module" = ESM | omitir = CommonJS
  "main": "src/index.js",      // Entry point para require() (CJS)
  "exports": {                 // Entry points para import (ESM) – mais preciso
    ".": ".src/index.js",
    "./utils": ".src/utils.js"
  },
  "bin": {
    "devtrack": ".cli.js"      // Cria o executável global "devtrack"
  },
  "scripts": {
    "start": "node src/index.js",
    "dev": "node --watch src/index.js", // Hot reload nativo Node 18+
    "test": "node --test",           // Test runner nativo Node 18+
    "lint": "eslint src/",
    "build": "node scripts/build.js"
  },
  "dependencies": {
    "commander": "^12.0.0",        // Produção: vai junto com o pacote
    "chalk": "^5.3.0"
  },
  "devDependencies": {
    "eslint": "^8.57.0"           // Dev: só para desenvolvimento
  },
  "peerDependencies": {
    "node": ">=18"                // Esperado no ambiente host
  },
  "engines": { "node": ">=18" },
  "files": ["src/", "cli.js"],    // Arquivos incluídos no npm publish
  "keywords": ["cli", "tasks"],  // Para busca no npm
  "license": "MIT"
}
```

8.1 — Versionamento Semântico (SemVer)

O SemVer define o significado de cada número da versão: **MAJOR.MINOR.PATCH**. Entender isso é essencial para evitar quebras inesperadas ao atualizar dependências.

Tipo	Quando usar	Exemplo	Compatível?
PATCH (0.0.X)	Bug fixes sem quebrar API	1.2.0 → 1.2.1	Sempre ✓
MINOR (0.X.0)	Nova feature, API retrocompatível	1.2.0 → 1.3.0	Geralmente ✓
MAJOR (X.0.0)	Breaking change — quebra API	1.2.0 → 2.0.0	Verificar ✗
^4.2.1	Aceita qualquer 4.x.x >= 4.2.1	Instalará 4.9.0	MINOR/PATCH
~4.2.1	Aceita qualquer 4.2.x >= 4.2.1	Instalará 4.2.9	Só PATCH

- [Documentação oficial — npm](#)
- [Documentação oficial — package.json](#)
- [SemVer — semver.org](#)

MÓDULO 9

CLI Profissional

Commander.js · Chalk · Ora · Inquirer.js

Com os fundamentos nativos dominados, as bibliotecas do ecossistema elevam o DevTrack a um produto profissional. Instale: `npm install commander chalk ora inquirer`

```
// Commander.js – Estrutura de comandos e flags
import { Command } from "commander";

const program = new Command()
  .name("devtrack")
  .description("CLI para gerenciamento de projetos")
  .version("1.0.0");

// Subcomando com argumento obrigatório e opções
program
  .command("add <titulo>") // <obrigatório> [opcional]
  .description("Adiciona uma nova tarefa")
  .option("-p, --prioridade <n>", "alta|media|baixa", "media")
  .option("-t, --tags <tags...>", "tags da tarefa")
  .option("-P, --projeto <nome>", "projeto associado")
  .action(async (titulo, opts) => {
    // opts.prioridade, opts.tags, opts.projeto
    await adicionarTarefa(titulo, opts);
  });

program.parse(process.argv);
// $ devtrack add "Implementar login" -p alta -t auth backend -P mvp
```



```
// Chalk 5+ é ESM puro – importe com import, não require
import chalk from "chalk";
import ora from "ora";

// Chalk – cores e estilos no terminal
console.log(chalk.green("✓ Tarefa criada com sucesso!"));
console.log(chalk.red.bold("✗ Erro ao salvar"));
console.log(
  chalk.cyan(`ID: ${id.slice(0,8)}`) +
  chalk.gray(` · ${new Date().toLocaleDateString("pt-BR")}`)
);

// Ora – spinner de progresso
const spinner = ora("Sincronizando com GitHub...").start();
try {
  await buscarIssuesGitHub(repo); // Módulo 5
  spinner.succeed(chalk.green("Sincronizado com sucesso!"));
} catch (err) {
  spinner.fail(chalk.red(`Erro: ${err.message}`));
}

// Inquirer – formulários interativos
import inquirer from "inquirer";

const respostas = await inquirer.prompt([
  { type: "input", name: "titulo",
    message: "Título da tarefa:",
    validate: v => v.length >= 3 || "Mínimo 3 caracteres" },
  { type: "list", name: "prioridade",
    message: "Prioridade:", choices: ["alta","media","baixa"] },
  { type: "checkbox", name: "tags",
    message: "Tags:", choices: ["frontend","backend","bug","feature"] },
  { type: "confirm", name: "confirmar",
    message: "Criar tarefa?", default: true },
]);
```

- [Documentação — Commander.js](#)
- [Documentação — Chalk](#)
- [Documentação — Ora](#)
- [Documentação — Inquirer.js](#)

Sintaxe de Classes e Estruturas Lineares

class · #private · static · Stack · Queue · Deque · LinkedList

10.1 — Sintaxe de Classes em JavaScript

JavaScript usa classes como **açúcar sintático** sobre protótipos. Não é necessário dominar toda a teoria de Orientação a Objetos agora, mas a sintaxe de classes é o veículo que usaremos para construir todas as estruturas de dados deste módulo em diante. Vamos percorrer cada elemento da sintaxe com exemplos progressivos.

[Anatomia completa de uma classe](#)

[illegible]

```
// ■■ Symbol.iterator – torna a classe usável em for...of ██████████  
[Symbol.iterator]() {  
    return this.#itens[Symbol.iterator]();  
}  
  
// ■■ Método estático – factory method ████████████████████████████████████████  
static fromArray(nome, arr) {  
    const col = new Colecao(nome);  
    arr.forEach(item => col.adicionar(item));  
    return col;  
}
```

Uma classe completa com todos os elementos de sintaxe: campos `#private`, `static`, `constructor`, `getters/setters`, `toString`, `Symbol.iterator` e `factory method`.

Uso práctico de todos os elementos

```

const frutas = new Colecao("Frutas", 3);

// Method chaining via return this
frutas.adicionar("maçã").adicionar("banana");
console.log(frutas.tamanho);    // 2 – via getter (sem parênteses)
console.log(frutas.cheio);      // false
console.log(`${frutas}`);       // "Colecao("Frutas", 2/3)" – via toString()

// Setter com validação
frutas.capacidade = 5;          // OK
// frutas.capacidade = 1;       // Error: Capacidade menor que o tamanho atual

// for...of via Symbol.iterator
for (const fruta of frutas) {
  console.log(fruta);           // "maçã", depois "banana"
}

// Spread operator também funciona via Symbol.iterator
const arr = [...frutas];        // ["maçã", "banana"]

// Campo privado: inacessível externamente
// frutas.#itens                 // SyntaxError

// Método estático: chamado na classe, não na instância
const legumes = Colecao.fromArray("Legumes", ["cenoura", "brócolis"]);
console.log(Colecao.totalCriadas); // 2 – conta todas as instâncias criadas

```

■ Output

```

2
false
Colecao("Frutas", 2/3)
maçã
banana
2

```

Herança com extends e super

JavaScript suporta herança de classes com `extends`. A classe filha herda todos os métodos e pode sobrescrever (override) os que precisar. `super()` chama o construtor da classe pai — **obrigatório** no construtor da classe filha antes de usar `this`.

```
// Classe base (pai)
class Estrutura {
  #nome;
  constructor(nome) { this.#nome = nome; }
  get nome() { return this.#nome; }
  isEmpty() { throw new Error("Implementar na subclasse"); }
  toString() { return `${this.#nome}(tamanho=${this.size})`; }
}

// Classe filha herda de Estrutura
class Stack extends Estrutura {
  #items = [];

  constructor() {
    super("Stack"); // OBRIGATÓRIO: chama constructor da classe pai
  }

  push(item) { this.#items.push(item); }
  pop() { return this.#items.pop(); }
  peek() { return this.#items.at(-1); }
  isEmpty() { return this.#items.length === 0; } // Override
  get size() { return this.#items.length; }
}

const s = new Stack();
s.push(10); s.push(20);
console.log(s.nome); // "Stack" — herdado do pai via getter
console.log(`${s}`); // "Stack(tamanho=2)" — toString() do pai
console.log(s instanceof Stack); // true
console.log(s instanceof Estrutura); // true — herança na chain
```

■ Output

```
"Stack"
"Stack(tamanho=2)"
true
true
```

instanceof verifica toda a cadeia de herança — s é instância tanto de Stack quanto de Estrutura.

■ Quer aprofundar em Protótipos e OOP?

As classes em JS são açúcar sobre prototype chain. Para entender o que acontece "por baixo" — `Object.getPrototypeOf`, prototype delegation, mixins — veja os links abaixo. O livro da Loiane também cobre esses conceitos nos capítulos iniciais. Para este workshop, a sintaxe acima é tudo que você precisa.

- [MDN — Classes em JavaScript \(referência completa\)](#)
- [MDN — Campos privados \(#\)](#)
- [MDN — Symbol.iterator](#)

→ [JavaScript.info — Classes](#) — Tutorial progressivo com exercícios interativos

→ [JavaScript.info — Herança com classes](#) — *extends*, *super*, *override* e *instanceof*

Classes de Erro Customizadas — Herança Prática

Criar classes de erro customizadas é um dos usos mais práticos de `extends`. Permite capturar tipos específicos de erro com `instanceof` e adicionar metadados (código HTTP, código de erro) ao erro.

```
// Classe base de erro do DevTrack
class DevTrackError extends Error {
  constructor(mensagem, codigo) {
    super(mensagem); // Passa mensagem para Error
    this.name = "DevTrackError";
    this.codigo = codigo;
    // Capturar stack trace corretamente em V8
    Error.captureStackTrace(this, this.constructor);
  }
}

// Erros específicos herdam de DevTrackError
class TarefaNaoEncontrada extends DevTrackError {
  constructor(id) {
    super(`Tarefa "${id}" não encontrada`, "TASK_NOT_FOUND");
    this.name = "TarefaNaoEncontrada";
    this.taskId = id;
  }
}

class ValidacaoError extends DevTrackError {
  constructor(campo, motivo) {
    super(`Campo "${campo}": ${motivo}`, "VALIDATION_ERROR");
    this.name = "ValidacaoError";
    this.campo = campo;
  }
}

// Uso: captura seletiva por tipo
try {
  const tarefa = await buscarTarefa("abc-123");
} catch (err) {
  if (err instanceof TarefaNaoEncontrada) {
    console.error(`404: ${err.message}`); // Tratamento específico
  } else if (err instanceof DevTrackError) {
    console.error(`Erro interno: ${err.codigo}`);
  } else {
    throw err; // Re-lança erros desconhecidos
  }
}
```

■ Output

404: Tarefa "abc-123" não encontrada

Erros customizados tornam o código de tratamento de erros muito mais legível e preciso do que comparar strings de mensagem.

Padrão de Método Abstrato — Contrato de Implementação

JavaScript não tem métodos abstratos nativos, mas podemos simular o contrato lançando um erro no método da classe base:

```
// "Abstrato": qualquer classe que herda DEVE implementar estes métodos
class EstruturaBase {
  // Métodos que obrigatoriamente devem ser implementados pelas subclasses
  add(item) { throw new Error(`${this.constructor.name} deve implementar add()`); }
  remove() { throw new Error(`${this.constructor.name} deve implementar remove()`); }
  peek() { throw new Error(`${this.constructor.name} deve implementar peek()`); }
  isEmpty() { throw new Error(`${this.constructor.name} deve implementar isEmpty()`); }
  get size() { throw new Error(`${this.constructor.name} deve implementar size`); }

  // Método concreto – usa os abstratos, funciona em qualquer subclasse
  toArray() {
    const resultado = [];
    while (!this.isEmpty()) resultado.push(this.remove());
    return resultado;
  }

  toString() {
    return `${this.constructor.name}(${this.size} itens)`;
  }
}

// Stack implementa o contrato – não vai lançar erros
class Stack extends EstruturaBase {
  #items = [];
  add(item) { this.#items.push(item); return this; }
  remove() { return this.#items.pop(); }
  peek() { return this.#items.at(-1); }
  isEmpty() { return this.#items.length === 0; }
  get size() { return this.#items.length; }
}

const s = new Stack();
s.add(1).add(2).add(3);
console.log(s.toString()); // "Stack(3 itens)"
console.log(s.toArray()); // [3, 2, 1] – método herdado funciona
```

■ Output

```
"Stack(3 itens)"
[3, 2, 1]
```

Checklist de Sintaxe de Classes

- **new NomeDaClasse()** — cria uma instância. Sem new → TypeError.

```
const s = new Stack(); // ✓ Stack(); //
                        // TypeError
```

• #campo — privado. Só acessível dentro da classe. SyntaxError se tentar fora.	<code>this.#items.push(x); // ✓ s.#items // SyntaxError</code>
• static método() — pertence à classe, não à instância.	<code>Stack.fromArray([1,2,3]); // ✓ s.fromArray // undefined</code>
• get prop() — lido como propriedade (sem parênteses).	<code>s.size // ✓ s.size() // TypeError: s.size não é função</code>
• super() no constructor — OBRIGATÓRIO antes de usar this em extends.	<code>constructor() { super(); this.x = 1; } // ✓</code>
• instanceof — verifica toda a cadeia de herança.	<code>s instanceof Stack // true s instanceof EstruturaBase // true</code>

10.2 — Stack (Pilha) — LIFO

Uma Stack segue o princípio **LIFO: Last In, First Out** — o último elemento inserido é o primeiro a ser removido. Analogia: uma pilha de pratos.

```
class Stack {
  #items = [];

  push(item) { this.#items.push(item); } // O(1)
  pop()      { return this.#items.pop(); } // O(1) – retorna undefined se vazia
  peek()    { return this.#items.at(-1); } // O(1) – vê o topo sem remover
  isEmpty() { return this.#items.length === 0; }
  get size() { return this.#items.length; }
  clear()   { this.#items = []; }
  toArray() { return [...this.#items]; }
}

// Aplicação no DevTrack: histórico de ações (undo/redo)
const historico = new Stack();

// Ao criar uma tarefa:
historico.push({ acao: "add", estado: JSON.stringify(tarefasAntes) });

// Ao desfazer (undo):
const ultimaAcao = historico.pop();
if (ultimaAcao) {
  const estadoAnterior = JSON.parse(ultimaAcao.estado);
  await salvarTarefas(estadoAnterior);
  console.log(chalk.yellow("Ação desfeita: " + ultimaAcao.acao));
}
```

■ **Loiane Groner** — Capítulo 3: Pilhas — estrutura LIFO, push, pop e casos de uso

10.3 — Queue (Fila) — FIFO

Uma Queue segue **FIFO: First In, First Out** — o primeiro a entrar é o primeiro a sair. A implementação ingênua com array e `shift()` é $O(n)$ para remoção. A implementação eficiente usa um objeto como "fita" com ponteiros de cabeça e cauda — ambas as operações em **$O(1)$** .

```
// Implementação O(1) com objeto + ponteiros
class Queue {
  #store = {}; // Armazena os itens por índice
  #head = 0; // Próximo item a sair
  #tail = 0; // Próxima posição livre

  enqueue(item) {
    this.#store[this.#tail] = item;
    this.#tail++;
  }

  dequeue() {
    if (this.isEmpty()) return undefined;
    const item = this.#store[this.#head];
    delete this.#store[this.#head]; // Liberar memória
    this.#head++;
    return item;
  }

  peek() { return this.#store[this.#head]; }
  isEmpty() { return this.#head === this.#tail; }
  get size(){ return this.#tail - this.#head; }
}

// Por que não usar Array.shift()?
// Array.shift() é O(n) — reindexa todos os elementos após remover o primeiro
// Com 10.000 itens: shift() reloca 9.999 posições a cada remoção
// Nossa Queue com objeto: dequeue() é O(1) — sempre constante

// Aplicação no DevTrack: fila de notificações/webhooks
const filaNotificacoes = new Queue();
filaNotificacoes.enqueue({ tipo: "task_created", taskId: "abc123" });
filaNotificacoes.enqueue({ tipo: "task_done", taskId: "def456" });

// Processar a fila
while (!filaNotificacoes.isEmpty()) {
  const evento = filaNotificacoes.dequeue();
  await enviarWebhook(evento); // Módulo 5: fetch
}
```

■ **Loiane Groner** — Capítulo 4: Filas e Deques — FIFO, enqueue, dequeue e eficiência

■ [MDN — Map](#) (atenção: diferente de `Array.map`)

■ [MDN — Symbol.iterator](#)

MÓDULO 11

Estruturas de Dados Não-Lineares

Trie · Graph (BFS/DFS) · Min Heap · LRU Cache · Map vs map

11.1 — Atenção: Map vs map

■ Map (estrutura) vs .map() (método de Array) — São coisas completamente diferentes!

`Array.prototype.map()` é um MÉTODO que transforma arrays: `[1,2,3].map(x => x*2) → [2,4,6]` Map (com M maiúsculo) é uma ESTRUTURA DE DADOS que mapeia chaves para valores, como um objeto, mas com chaves de qualquer tipo e com tamanho rastreado automaticamente. Use o link abaixo para entender as diferenças vs Object: quando usar cada um.

```
// Map – estrutura chave:valor (como um dicionário)
const grafo = new Map();           // Chaves podem ser qualquer tipo!
grafo.set("A", ["B", "C"]);        // chave: string, valor: array
grafo.set("B", ["D"]);
grafo.get("A");                     // => ["B", "C"]
grafo.has("X");                     // => false
grafo.size;                         // => 2 (propriedade, não método)
grafo.delete("B");

for (const [chave, valor] of grafo) { // Iterável com ordem de inserção
  console.log(chave, valor);
}

// [1,2,3].map() – MÉTODO de array – transforma elementos
const dobrado = [1, 2, 3].map(x => x * 2); // => [2, 4, 6]
```

11.2 — Trie — Busca e Autocompletar

Uma Trie (prefixo-árvore) é uma estrutura especializada para strings. Inserção e busca por prefixo têm complexidade **O(m)** onde m é o tamanho da string — independente do número de palavras armazenadas.

```

class TrieNode {
  constructor() {
    this.filhos = new Map(); // char → TrieNode
    this.fimDaChave = false;
    this.taskId = null;      // ID da tarefa (na folha)
  }
}

class Trie {
  #raiz = new TrieNode();

  inserir(palavra, id = null) {
    let no = this.#raiz;
    for (const ch of palavra.toLowerCase()) {
      if (!no.filhos.has(ch)) no.filhos.set(ch, new TrieNode());
      no = no.filhos.get(ch);
    }
    no.fimDaChave = true;
    no.taskId = id;
  }

  buscarPrefixo(prefixo) {
    let no = this.#raiz;
    for (const ch of prefixo.toLowerCase()) {
      if (!no.filhos.has(ch)) return []; // Prefixo não existe
      no = no.filhos.get(ch);
    }
    return this.#coletar(no, prefixo.toLowerCase());
  }

  #coletar(no, prefixo) {
    const resultado = [];
    if (no.fimDaChave) resultado.push({ palavra: prefixo, id: no.taskId });
    for (const [ch, filho] of no.filhos)
      resultado.push(...this.#coletar(filho, prefixo + ch));
    return resultado;
  }
}

// Uso no DevTrack – autocompletar títulos de tarefas
const trie = new Trie();
["Fix login bug", "Fix CSS", "Feature auth", "Add tests"].forEach(
  (t, i) => trie.inserir(t, `task-${i}`)
);
trie.buscarPrefixo("Fix");

```

■ Output

```

[
  { palavra: "fix login bug", id: "task-0" },

```

```
{ palavra: "fix css", id: "task-1" }  
]
```

■ **Loiane Groner** — Capítulo 8: Árvores — estruturas hierárquicas, BST e travessias

11.3 — Graph — Dependências entre Tarefas

Um Grafo (Graph) modela relacionamentos entre entidades. Grafo **dirigido** (as arestas têm direção) é ideal para dependências: "Tarefa A depende de B".

```
class Graph {  
  #adj = new Map(); // Mapa de adjacência: vertice → [vizinhos]  
  
  addVertex(v)      { if (!this.#adj.has(v)) this.#adj.set(v, []); }  
  addEdge(u, v)     { this.#adj.get(u)?.push(v); }  
  neighbors(v)      { return this.#adj.get(v) ?? []; }  
  
  // BFS — busca em largura (nível por nível)  
  bfs(inicio) {  
    const visitados = new Set([inicio]);  
    const fila = [inicio];  
    const ordem = [];  
    while (fila.length) {  
      const v = fila.shift();  
      ordem.push(v);  
      for (const viz of this.neighbors(v))  
        if (!visitados.has(viz)) { visitados.add(viz); fila.push(viz); }  
    }  
    return ordem;  
  }  
  
  // Detectar ciclo via DFS com coloração (branco/cinza/preto)  
  temCiclo() {  
    const COR = { BRANCO: 0, CINZA: 1, PRETO: 2 };  
    const cores = new Map([...this.#adj.keys()].map(v => [v, COR.BRANCO]));  
  
    const dfs = (v) => {  
      cores.set(v, COR.CINZA);  
      for (const viz of this.neighbors(v)) {  
        if (cores.get(viz) === COR.CINZA) return true; // Ciclo!  
        if (cores.get(viz) === COR.BRANCO && dfs(viz)) return true;  
      }  
      cores.set(v, COR.PRETO);  
      return false;  
    };  
  
    return [...this.#adj.keys()].some(v => cores.get(v) === COR.BRANCO && dfs(v));  
  }  
}
```

11.4 — LRU Cache — Cache com Política de Evicção

Um LRU (Least Recently Used) Cache guarda os N itens mais recentemente acessados. Quando o cache está cheio e precisa inserir um novo item, descarta o **menos recentemente utilizado**. Implementação com Map: $O(1)$ para get e set.

```

// Map em JavaScript preserva a ORDEM de inserção
// Isso nos permite usar um único Map como LRU simples
class LRUCache {
  #cache;
  #cap;

  constructor(capacidade) {
    this.#cap = capacidade;
    this.#cache = new Map();
  }

  get(chave) {
    if (!this.#cache.has(chave)) return undefined;
    const valor = this.#cache.get(chave);
    // Mover para o "fim" (mais recente): delete + set
    this.#cache.delete(chave);
    this.#cache.set(chave, valor);
    return valor;
  }

  set(chave, valor) {
    if (this.#cache.has(chave)) this.#cache.delete(chave);
    else if (this.#cache.size >= this.#cap) {
      // Evictar o PRIMEIRO item (menos recente)
      const maisAntigo = this.#cache.keys().next().value;
      this.#cache.delete(maisAntigo);
    }
    this.#cache.set(chave, valor);
  }

  get size() { return this.#cache.size; }
}

// Uso no DevTrack: cache de respostas da API GitHub (Módulo 5)
const cache = new LRUCache(50);
async function fetchComCache(url) {
  const cached = cache.get(url);
  if (cached) return cached; // Cache hit
  const resp = await fetch(url); // Cache miss
  const dados = await resp.json();
  cache.set(url, dados);
  return dados;
}

```

■ **Loiane Groner** — Capítulo 7: Dicionários e Hashes — HashMap e operações O(1)

■ [MDN — Map](#)

■ [MDN — Set](#)

MÓDULO 12

Performance e Segurança

`worker_threads` · `perf_hooks` · `crypto` (hash · HMAC · criptografia simétrica)

O último módulo cobre os dois pilares de sistemas profissionais: **performance** (usar múltiplos núcleos da CPU com Workers e medir com precisão) e **segurança** (hash, HMAC, criptografia e geração de tokens seguros).

12.1 — `worker_threads` — Paralelismo Real com Múltiplas Threads

O Node é single-threaded por padrão, mas `worker_threads` cria threads reais que executam código JavaScript em paralelo. Ideal para operações CPU-intensivas que travariam a thread principal.

```

// main.js – Thread principal
import { Worker, isMainThread, workerData, parentPort } from "worker_threads";
import os from "os"; // Módulo 6

// Criar um pool de workers limitado ao número de CPUs
async function processarEmParalelo(lotes) {
  const maxWorkers = os.cpus().length;
  const resultados = [];

  // Processar em batches para não criar mais workers que CPUs
  for (let i = 0; i < lotes.length; i += maxWorkers) {
    const batch = lotes.slice(i, i + maxWorkers);
    const promises = batch.map(lote => executarWorker(lote));
    resultados.push(...await Promise.all(promises)); // Módulo 2
  }
  return resultados;
}

function executarWorker(dados) {
  return new Promise((resolve, reject) => {
    const w = new Worker(new URL("./worker.js", import.meta.url), {
      workerData: dados
    });
    w.on("message", resolve);
    w.on("error", reject);
    w.on("exit", code => {
      if (code !== 0) reject(new Error(`Worker saiu com código ${code}`));
    });
  });
}

// worker.js – Executado na thread filha
import { workerData, parentPort } from "worker_threads";

const { tarefas } = workerData;
const resultado = tarefas.map(t => ({ id: t.id, score: calcularScore(t) }));
parentPort.postMessage(resultado); // Envia de volta para a thread principal

```

12.2 — perf_hooks — Medir Performance com Precisão

```
import { performance, PerformanceObserver } from "perf_hooks";

// Método 1: marks e measures
performance.mark("inicio-trie");
const resultados = trie.buscarPrefixo("Fix");
performance.mark("fim-trie");
performance.measure("busca-trie", "inicio-trie", "fim-trie");

const [medida] = performance.getEntriesByName("busca-trie");
console.log(`Trie: ${medida.duration.toFixed(4)}ms`);

// Método 2: performance.now() — simples e direto
const inicio = performance.now();
const resultArray = tarefas.filter(t => t.titulo.startsWith("Fix"));
const tempo = performance.now() - inicio;
console.log(`Array.filter: ${tempo.toFixed(4)}ms`);
```

12.3 — Fundamentos de Criptografia no Módulo crypto

Criptografia é a ciência de transformar informação em um formato ilegível para quem não possui a chave. Existem dois tipos principais:

- **Criptografia simétrica:** A mesma chave cifra e decifra. Rápida, ideal para grandes volumes de dados. Algoritmo mais usado: AES-256-GCM.
- **Criptografia assimétrica:** Duas chaves — pública (cifra) e privada (decifra). Usada em TLS/HTTPS e assinatura digital. Algoritmos: RSA, Ed25519.
- **Hash:** Função de mão única — não é criptografia! Impossível (na prática) reverter. SHA-256 é o padrão para integridade de dados.
- **HMAC:** Hash com chave secreta para autenticar mensagens — garante que a mensagem não foi alterada.


```
const chave = crypto.scryptSync("minha-senha", "salt-unico", 32);

// Uso
const mensagem = "dados secretos do DevTrack";
const pacote = cifrar(mensagem, chave);
const recuperado = decifrar(pacote, chave);
console.log(recuperado); // => "dados secretos do DevTrack"
```

AES-256-GCM é o modo autenticado — garante confidencialidade E integridade. Nunca reutilize o IV (Initialization Vector) com a mesma chave.

■ Nunca use `Math.random()` para segurança

`Math.random()` não é criptograficamente seguro — é previsível. Para tokens, senhas, chaves ou qualquer dado sensível, use sempre `crypto.randomBytes()` ou `crypto.randomUUID()`. A diferença: `Math.random()` é para jogos e simulações; `crypto` é para segurança.

■ [Documentação oficial — worker_threads](#)

■ [Documentação oficial — perf_hooks](#)

■ [Documentação oficial — crypto](#)

→ [Cryptography Explained \(Computerphile, YouTube\)](#) — Vídeo visual sobre criptografia simétrica/assimétrica

→ [OWASP Cryptographic Storage Cheat Sheet](#) — Referência profissional para uso seguro de cripto

■ [Loiane Groner](#) — Capítulo 12: Algoritmos Avançados — análise de complexidade e otimização

Conclusão e Próximos Passos

Você chegou ao final do Workshop Node.js. Ao longo dos 12 módulos, percorreu uma jornada que vai do entendimento do Event Loop até criptografia simétrica — passando por assincronicidade, I/O de arquivos, rede, processos, streams, gerenciamento de pacotes, CLI profissional e as principais estruturas de dados. Cada módulo foi construído sobre o anterior, e esse conhecimento se conecta.

O que você é capaz de fazer agora

Área	Capacidade adquirida
Runtime	Explicar o Event Loop, entender a ordem de execução e diagnosticar comportamentos assíncronos
Async	Escrever código assíncrono com callbacks, Promises e async/await; tratar erros corretamente
I/O	Ler, escrever e monitorar arquivos de forma eficiente com fs/promises e streams
Rede	Criar servidores HTTP, consumir APIs externas, entender os conceitos de DNS/URL/HTTP
CLI	Construir ferramentas de linha de comando profissionais com Commander, Chalk e Inquirer
Estruturas	Implementar Stack, Queue, Trie, Graph e LRU Cache do zero com complexidade correta
Segurança	Gerar tokens seguros, fazer hash de arquivos, autenticar mensagens com HMAC e cifrar dados
Performance	Medir e comparar algoritmos com perf_hooks e usar worker_threads para paralelismo

Sequência de Aprendizagem Recomendada — O que Estudar a Seguir

Imediato

- Completar a Apostila de Tarefas — construir o DevTrack do zero ao fim
- Ler os capítulos indicados do livro da Loiane Groner para consolidar as estruturas de dados
- Experimentar o Node 22 LTS: --watch mode nativo, Test Runner, Permission Model

Curto prazo

- TypeScript — adiciona tipos estáticos ao JavaScript; detecta erros antes de executar
- Express.js ou Fastify — frameworks HTTP para construir APIs REST completas
- Prisma + PostgreSQL — banco de dados relacional com ORM moderno

Médio prazo

- Testes automatizados — Vitest ou Jest para unit tests; Supertest para API tests
- Docker — containerizar o DevTrack para deploy reproduzível

- CI/CD com GitHub Actions — executar testes automaticamente em cada push

Aprofundamento

- Arquitetura de sistemas: Event-Driven Design, CQRS, Circuit Breaker
- Node.js em produção: PM2, clustering, graceful shutdown, health checks
- Algoritmos e complexidade: Big O na prática, Dynamic Programming, grafos avançados

Referências Consolidadas

Todas as fontes utilizadas neste workshop, organizadas por área:

Runtime e Linguagem

- [Node.js Docs](#)
- [MDN Web Docs](#)
- [JavaScript.info](#)
- [Node.js Best Practices](#)

Estruturas de Dados

- [Loiane Groner — Estruturas de Dados e Algoritmos com JavaScript](#)
- [Visualgo — Visualizador de algoritmos](#)

Segurança

- [OWASP Node.js Security Cheat Sheet](#)
- [Cryptography Explained \(Computerphile\)](#)

Redes

- [MDN — Como a Web funciona](#)
- [Khan Academy — How the Internet Works](#)

Este material é um ponto de partida — não um destino. O melhor aprendizado vem de construir, quebrar, entender o erro e construir de novo. A Apostila de Tarefas foi projetada exatamente para isso: um projeto real, com código real, que cresce módulo a módulo.